1. Why Do We Need Tables?

In the previous class, we accessed Delta data using a physical path.

```
spark.read \
    .format("delta") \
    .load("<path-to-orders-delta>")
```

This works for a few datasets.

But in a real project, we may have:

- Orders
- Customers
- Products
- Payments
- Inventory
- Hundreds of other datasets

Remembering every physical storage path is not practical.

Instead of:

/path/project/ecommerce/orders/

we want to write:

SELECT * FROM orders;

For that to work, Databricks needs information about what `orders` means.

That information is **metadata**.

---

2. Data vs Metadata

Data

Data means the actual business records.

Example:

1001,C101,P101,2026-09-01,2,899.00,COMPLETED

This is an actual order record.

Metadata

Metadata means **information about the data**.

For an orders dataset:

Table Name → orders
Format → Delta
Columns → order_id, customer_id, product_id...
Location → Physical storage path

Remember

DATA
→ Actual records

METADATA
→ Information describing those records

Metadata does not replace actual data. It describes and helps manage it.

---

3. Why Metadata Is Required

Without metadata, we would need to manually remember:

- What datasets exist
- What each dataset is called
- Which columns exist
- Data types
- Storage format
- Physical location

With metadata, we can work using logical names such as:

orders
customers
products
payments

instead of raw storage paths.

---

4. What Is a Metastore?

A **metastore** manages metadata about data objects.

It helps the processing/query engine understand:

- What tables exist
- What they are called
- Their schemas
- Their formats
- Where their data is stored

Simple view:

Actual Data
↓
Storage

Metadata
↓
Metastore

Important:

> A metastore does not contain all your business records as metadata.

It manages information **about** those data objects.

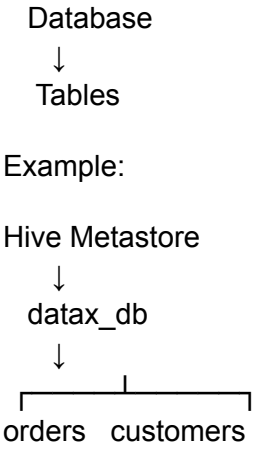
---

5. What Is Hive Metastore?

Hive Metastore is the traditional metadata system used in Databricks environments for managing table metadata.

In this class, the hierarchy is:

Hive Metastore
↓



---

6. What Is a Database?

A database is a **logical container for related tables**.

It does not mean all table data is stored inside one database file.

Example:

datax_db

can logically contain:

orders
customers
products

while each table's actual data is persisted in storage.

---

7. Exploring Databases

To view available databases:

SHOW DATABASES;

Different environments may show different results.

A common default database is:

default

---

8. Creating a Database

Create the project database:

CREATE DATABASE IF NOT EXISTS datax_db;

Meaning:

CREATE DATABASE
→ Create a logical database

IF NOT EXISTS
→ Avoid failure if it already exists

datax_db
→ Database name

Validate:

SHOW DATABASES;

---

9. Selecting a Database

USE datax_db;

Check the currently selected database:

SELECT current_database();

Expected:

datax_db

Once selected, we can often refer to tables without writing the full database name every time.

---

10. Delta Data at a Path vs a Table

This distinction is very important.

Previously:

```
orders_df.write \
   .format("delta") \
   .save(delta_path)
```

This persisted Delta data at a path.

But it did **not necessarily create**:

datax_db.orders

Remember

Delta Data at Path
≠
Registered Table

They are related, but they are not the same concept.

---

11. Creating a Managed Delta Table

Suppose we already have:

orders_df

Create a table using:

```
orders_df.write \
   .format("delta") \
   .mode("overwrite") \
   .saveAsTable("datax_db.orders")
```

Meaning

.format("delta")

Store the table in Delta format.

.mode("overwrite")

Used here for a repeatable classroom demo.

.saveAsTable("datax_db.orders")

Persist and register the DataFrame as a named table.

---

12. Validate the Table

Check tables:

SHOW TABLES IN datax_db;

Expected:

orders

Query it:

SELECT *
FROM datax_db.orders;

Now compare:

Earlier:

spark.read.format("delta").load("<physical-path>")

Now:

SELECT *
FROM datax_db.orders;

Main Improvement

We now use a **logical table name** instead of manually passing a physical path.

---

13. Database + Table Naming

A fully qualified Hive Metastore table reference in this class is:

datax_db.orders

Breakdown:

datax_db
↓
Database

orders
↓
Table

Pattern:

database.table

If you already ran:

USE datax_db;

then:

SELECT * FROM orders;

can resolve to the same table.

---

14. Table vs DataFrame

Do not confuse:

orders_df

with:

datax_db.orders

DataFrame

orders_df

- Spark processing abstraction
- Used inside code
- Exists programmatically in the Spark session/context

Table

datax_db.orders

- Registered table
- Has persistent metadata
- Can be referenced using table name
- Data is persisted in storage

Easy Way to Remember

DataFrame
→ Used to process data

Table
→ Gives persisted data a named, queryable identity

---

15. Where Is the Actual Table Data?

A table name does not mean that the business data lives inside the metastore.

Inspect:

DESCRIBE EXTENDED datax_db.orders;

You may see information such as:

- Provider
- Location
- Type
- Columns

The relationship is:

datax_db.orders
↓
Metadata
↓
Physical Delta Data

The table metadata helps Databricks locate and understand the actual stored data.

---

16. What Is a Managed Table?

At a beginner level, a managed table is a table where:

- We did not manually specify an external `LOCATION`.
- The platform/metastore manages the table storage location.
- Metadata and managed data lifecycle are tied together.

We created:

.saveAsTable("datax_db.orders")

without writing:

LOCATION '/some/custom/path'

So this is our first **managed table**.

---

17. Managed vs External — Basic Idea

At this stage, only remember the distinction.

Managed Table

Metastore manages metadata
+
Platform manages table data lifecycle/location

External Table

Metastore manages metadata
+
Data exists at a location explicitly controlled outside

The detailed behavior is covered in the next class.

---

18. Inspecting Table Metadata

Check table structure:

DESCRIBE TABLE datax_db.orders;

This shows:

- Columns
- Data types

Example:

order_id
customer_id
product_id
order_date
quantity
unit_price
order_status

This is not actual order data.

It describes the structure.

For more details:

DESCRIBE EXTENDED datax_db.orders;

This may show:

- Columns
- Provider
- Location
- Table properties/type information

---

19. The Complete Hierarchy

Before:

Physical Path
↓
Delta Files

After introducing metadata:

Hive Metastore
↓
datax_db
↓
orders
↓
Delta Data

This is the main transition of this class.

---

20. Does Hive Metastore Store Actual Orders?

No.

The Hive Metastore manages **metadata**.

The actual business records are persisted in storage.

Hive Metastore
↓
Metadata

versus:

Storage
↓
Actual Data

This distinction is essential.

---

21. Common Errors

Database Not Found

Possible error:

SCHEMA_NOT_FOUND

Fix:

CREATE DATABASE IF NOT EXISTS datax_db;

Table Not Found

You run:

SELECT * FROM orders;

but the current database is different.

Fix:

USE datax_db;

Then:

SELECT * FROM orders;

Or use the fully qualified name:

SELECT *
FROM datax_db.orders;

Table Already Exists

A write may fail if the table already exists and no appropriate mode is supplied.

For classroom practice:

.mode("overwrite")

is used.

But `overwrite` should be used carefully in real pipelines.

---

22. End-to-End Flow

orders.csv
↓
Spark DataFrame
↓
orders_df
↓
Write as Delta Table
↓
datax_db.orders

And metadata hierarchy:

Hive Metastore
↓
datax_db
↓
orders

---

23. Quick Revision

DATA

Actual business records.

METADATA

Information describing data and data objects.

METASTORE

Manages metadata.

DATABASE

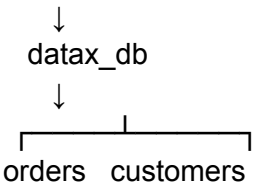
Logical container for tables.

TABLE

Named and queryable representation of persisted data.

Current Hierarchy

Hive Metastore



---

24. Student Practice

Use the `customers.csv` from the previous class.

Create:

datax_db.customers

Step 1: Read CSV

```
customers_df = (
    spark.read
    .option("header", True)
    .option("inferSchema", True)
    .csv("<customers-file-path>")
)
```

Step 2: Display

```
display(customers_df)
```

Step 3: Save as Managed Delta Table

```
customers_df.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("datax_db.customers")
```

Step 4: Validate

```
SHOW TABLES IN datax_db;
```

Expected tables:

orders
customers

Step 5: Query

```
SELECT *
FROM datax_db.customers;
```

Step 6: Inspect Metadata

```
DESCRIBE EXTENDED datax_db.customers;
```

Identify:

- Table name
- Columns
- Data types
- Provider
- Location

---

25. Interview-Relevant Questions

1. What is metadata?

Metadata is information about data, such as table name, columns, schema, storage format, and physical location.

---

2. What is a metastore?

A metastore manages metadata about tables and related data objects.

---

3. Does Hive Metastore store actual business records?

No.

It manages metadata, while actual records are persisted in storage.

---

4. What is a database in Hive Metastore?

A database is a logical container or namespace used to organize related tables.

---

5. What is the difference between a Delta path and a registered table?

Delta data at a path is persisted physical data.

A registered table adds a logical name and persistent metadata that allows the dataset to be queried by name.

---

6. What is the difference between a DataFrame and a table?

A DataFrame is a Spark processing abstraction.

A table is a named, queryable representation of persisted data with registered metadata.

---

7. What does `saveAsTable()` do?

It persists the DataFrame and registers it as a named table.

---

8. What is a managed table?

At a beginner level, it is a table where the platform manages the table's storage location and data lifecycle along with its metadata.

---

9. What is the difference between `DESCRIBE TABLE` and actual table data?

`DESCRIBE TABLE` shows metadata such as columns and types.

It does not return the actual business records.

---

10. Why are table names better than manually using physical paths everywhere?

Table names provide a logical, easier-to-manage abstraction over physical storage and allow users to query datasets without remembering exact storage locations.